

Software Dev. & Problem Solving II

GCIS-124

Multi-Threaded Server and
Multi-Threaded Client GUI

Activity Session

Goals of the Assignment

In this session you will create a small chat server and client. The system will allow multiple users to connect to a common server and chat in a public format, similar to a public channel on Discord. Both the server and the client provide an opportunity to use networking in conjunction with threading.

Activities

1. The tinchat system uses a very simple protocol consisting of the following commands.

Client Request	Server Response
<Name>	Connected
<Message>	<Name>: <Message> Sent to all clients, including <Name>: <Connected> <Name>: <Disconnected>
Disconnect	Disconnected

2. Start by creating a `ClientHandler` class that extends `Duplexer` and implements `Runnable`. This will be the class that handles a specific client's requests and broadcasts messages to all clients.
 - a. What fields are needed?
 - b. What parameters will the constructor need?
 - c. It should handle the server side of the protocol listed in **step 1**.
 - d. Consider a helper method to broadcast every message to all clients.
 - e. For debugging purposes, it should print all messages that are received in the format `<Name>: <Message>`.
 - f. Ensure that any shared resources are only able to be read or updated by one thread at a time.
3. Next, create a multithreaded `TinyChatServer`.

- a. The server should listen for client connections using TCP/IP on port 54321.
 - b. Each time a connection is accepted
 - i. Create a new `ClientHandler` and pass it the socket and list of clients
 - ii. Create a new `Thread` with the `ClientHandler` and start it. This allows the server to continue accepting connections.
4. A `TinyChatClientGUI` boilerplate class has been provided to you.
 - a. The overall design is up to you but should have the following controls. Take a moment to sketch out the layout of your GUI. Just do a basic GUI for now; we can improve the aesthetics later.
 - i. `TextField` - for the user to enter their name.
 - ii. `Button` - to connect to the server.
 - iii. `Button` - to disconnect from the server.
 - iv. `Label` - to display messages from this user and all users.
 - v. `TextField` - for the user to enter their message.
 - vi. `Button` - to send a message (optional; you can also send a message when the user hits enter after typing the message).
 - b. First, we'll need to connect to the server when the `Connect Button` is clicked.
 - i. Create a method to handle the connection to the server.
 - ii. Use a lambda function to call the connect method from the `Connect Button`'s `setOnAction` method.
 - iii. Create a new `Duplexer` and connect to the server
 - iv. Send the user's name and validate the appropriate server response.
 - v. In order to update the client with messages from other clients, it must always be listening for new messages. This is known as *asynchronous communication* because the client does not ask the server for new messages first, instead they simply arrive at any point in time.
 1. Create a separate thread to receive messages.
 2. When a message is received, we will need to use the JavaFX method `Platform.runLater` to update the Messages Label


```
Platform.runLater(() -> {
                    // Code to update messages label
                });
```
 3. The thread should run until it receives the `Disconnected` message from the server.
 - c. Create a method for when the `Disconnect Button` is clicked
 - i. Create a method to disconnect from the server.
 - ii. Use a lambda function to call the disconnect method from the `Button`'s `setOnAction` method.
 - d. Lastly, we need to send the `TextField` text to the server after a message is typed and enter is pressed or the `Send Button` is clicked. This code is small enough that it can be done in a lambda function without a separate method.
5. Test your client and server basic functionality
 - a. Start the server.
 - b. Start the client GUI.

- c. Enter your name and click the `Connect Button`.
 - d. Send multiple messages to the server and verify they are being received by the server and are displayed in the `Message Label` in the GUI.
 - e. Click the `Disconnect Button` and validate the client disconnects.
6. Test your client and server with multiple clients.
 - a. Ensure that all messages are seen by all clients.
7. Improve the user experience
 - a. Disable the `Message TextField` and `Disconnect Button` until the user is connected
 - i. Once connected, disable the `Connect Button`, and enable the `Message TextField` and `Disconnect Button`.
 - ii. When the user disconnects, disable the `Message TextField` and `Disconnect Button`, and enable the `Connect Button`.
 - b. Ensure that if the window is resized, there is no empty space, i.e. the GUI controls grow so that no empty space occurs.
 - c. Add prompts to the `TextField` controls.
 - d. Clear the `message TextField` after the message has been sent.
 - e. Enhance with colors and fonts.
 - f. Any other creative ideas?

Compiling and Running your GUI from the command line

- Your command line is not aware of the JavaFX libraries (jars) or vmargs that you specified in VSCode. When you run `javac` and `java` from the command line, you will need to use special command line options to let `javac` and `java` know about JavaFX
 - Classpath - a list of libraries (with absolute path) that are needed to compile your classes. The libraries (with absolute path) are separated by a semicolon (“;”). It is common to include your current directory (“.”) as the start.
 - Module-path - the path to the modules listed in the `--add-modules` option. This is the absolute path, and only the absolute path, to the libraries folder.
 - Add-modules - modules to use when running the application. These are the names of the libraries without the “.jar” extension.

```
javac -classpath
".;<absolute_path>\javafx.base.jar;<absolute_path>\javafx.controls.jar;
<absolute_path>\javafx.fxml.jar;<absolute_path>\javafx.graphics.jar"
*.java
```

```
java --module-path "<absolute_path>" --add-modules
javafx.controls,javafx.fxml,javafx.media TinyChatGUI
```